

Agreeo Technical Appendix: Complete Cypher Query Guide

Line-by-Line Technical Analysis of All 24 Presentation Queries

Document Type: Production Architecture Technical Reference & Exam Guide

Target Audience: Engineering Faculty, Systems Architects, Database Specialists

System: Agreeo — Graph-Native Movie Recommendation & Social Decision Engine

Core Engine: Neo4j Graph Database 5.x, Cypher Query Language, APOC Core, Vector Search (HNSW)

Executive Summary & Architecture Map

This document provides an exhaustive, line-by-line architectural breakdown of the **24 Cypher queries** featured in the official Technical Appendix of the Agreeo presentation.

In traditional relational architectures (RDBMS), calculating multi-hop collaborative filtering, real-time group taste intersections, and vector-semantic retrieval requires expensive, multi-table Cartesian joins ($O(N^k)$) that choke database memory under scale. Agreeo leverages **Index-Free Adjacency (IFA)**: relationships in Neo4j are physical double-linked pointers stored directly on disk blocks, allowing edge traversal in constant time $O(1)$ per hop regardless of global graph size.

The 24 queries in this appendix are organized across 11 core functional slides:

Slide	Functional Theme	Query Count	Key Operations Covered
01	System Context & Vision	2	Topology inspection (<code>apoc.meta.stats</code>), dual-population census
02	Data Model & Schema Boundary	2	Deterministic TMDb/MovieLens bridge (<code>I08</code>), DDL schema constraints
03	Ingestion & Idempotence	2	Micro-batched rating ETL (<code>I09</code>), non-destructive upsert & edge pruning (<code>M01</code>)
04	Collaborative Filtering	1	Master 4-hop collaborative traversal with mathematical ranking (<code>M23</code>)
05	Hybrid Retrieval	3	HNSW vector index DDL, semantic mood search (<code>M24</code>), Reciprocal Rank Fusion (RRF)
06	Transactions & Integrity	3	Atomic daily quota increment (<code>M06</code>), mutual exclusion cleanup (<code>M08</code>), edge creation (<code>M09</code>)
07	Social Graph Invariants	2	Atomic mutual friendship acceptance (<code>S06</code>), directional request creation (<code>S01</code>)
08	Group Workflow & Concurrency	2	Multi-user taste matrix (<code>S40</code>), pessimistic lock on event node for voting (<code>S44</code>)
09	Query Engineering	2	Subquery isolation anti-Cartesian pattern (<code>M22</code>), Cartesian explosion anti-pattern
10	Security & Trade-offs	2	100% parameterized execution against injection, defensive UNWIND pattern
11	Live Demo & Proofs	3	Live bridge inspection (<code>P05</code>), <code>PROFILE</code> execution plan (<code>P03</code>), degree centrality (<code>P04</code>)

Slide 01: System Context & Vision

Query 01 — APOC Graph Sizing & Density Inspection

- **Appendix Reference:** `System Verification`
- **Slide:** 01 · System Context & Vision
- **Execution Frequency:** Offline diagnostics, smoke testing, CI/CD verification
- **Key Goal:** Extract instant structural metadata proving graph density and relationship scaling without table scans.

```
// Verify topology, node labels, and relationship density
CALL apoc.meta.stats()
YIELD nodeCount, relCount, relTypeCount, labelCount
RETURN nodeCount, relCount, relTypeCount, labelCount;
```

Line-by-Line Breakdown:

1. `// Verify topology, node labels, and relationship density` * Descriptive comment declaring the intent: evaluating database sizing and graph topology density.

2. `CALL apoc.meta.stats()` * Invokes the APOC (Awesome Procedures on Cypher) internal metadata procedure. * *Mechanism:* Rather than executing an exhaustive traversal ($O(V + E)$), `apoc.meta.stats()` reads pre-aggregated transaction counters directly from Neo4j's transactional store header (`neostore.*`). Execution time is instantaneous ($O(1)$) even on graphs containing hundreds of millions of nodes.
3. `YIELD nodeCount, relCount, relTypeCount, labelCount` * Selects specific output variables emitted by the procedure stream into the Cypher pipeline. * `nodeCount` : Total active physical vertex records. * `relCount` : Total active physical relationship records (directed pointers). * `relTypeCount` : Number of distinct relationship types registered in the schema (e.g., `LIKED`, `RATED`, `FRIEND`). * `labelCount` : Number of distinct node labels registered (e.g., `AppUser`, `Movie`, `Genre`).
4. `RETURN nodeCount, relCount, relTypeCount, labelCount;` * Projects the extracted metrics to the calling client driver for assert verification.

Architectural Significance:

- **Density Metric:** Graph density is calculated as $\text{Density} = \frac{2 \times \text{relCount}}{\text{nodeCount} \times (\text{nodeCount} - 1)}$. In Agreeo, `relCount` exceeds `nodeCount` by orders of magnitude due to collaborative ratings and social links, proving the graph model fits the workload far better than relational tables.

Query 02 — Dual-Population Graph Census

- **Appendix Reference:** `Architecture Check`
- **Slide:** 01 · System Context & Vision
- **Execution Frequency:** Sanity check, monitoring dashboards, migration verification
- **Key Goal:** Verify clean separation between authenticated application users (`AppUser`) and historical analytical rating seeds (`MovieLensUser`).

```
// Dual-population census: active authenticated users vs analytical seed users
MATCH (u:AppUser) WITH count(u) AS appUsers
MATCH (ml:MovieLensUser) WITH appUsers, count(ml) AS mlUsers
MATCH (m:Movie) WITH appUsers, mlUsers, count(m) AS canonicalMovies
RETURN appUsers, mlUsers, canonicalMovies;
```

Line-by-Line Breakdown:

1. `// Dual-population census: active authenticated users vs analytical seed users` * Comment establishing the architectural purpose: verifying population partitioning.
2. `MATCH (u:AppUser) WITH count(u) AS appUsers` * Matches all nodes with the label `:AppUser`. The Cypher query planner resolves `count(u)` directly from the count-store metadata ($O(1)$). * `WITH count(u) AS appUsers` isolates the aggregate, transforming the result stream into a single scalar row and preventing Cartesian record generation in downstream clauses.
3. `MATCH (ml:MovieLensUser) WITH appUsers, count(ml) AS mlUsers` * Matches historical seed users from MovieLens 100k/1M. * Aggregates with `count(ml)`, forwarding both `appUsers` and `mlUsers` as a single 1-row record.
4. `MATCH (m:Movie) WITH appUsers, mlUsers, count(m) AS canonicalMovies` * Matches canonical application movies (`:Movie`), counting total playable records.
5. `RETURN appUsers, mlUsers, canonicalMovies;` * Emits the 3-element verification tuple.

Architectural Significance:

- **The Dual-Population Invariant:** Operational accounts (`AppUser`) have credentials, JWTs, and dynamic daily quotas. Seed accounts (`MovieLensUser`) are immutable analytical nodes that never authenticate. Mixing them into a single label would pollute indexes and introduce security liabilities. This query guarantees the boundary is preserved.

Slide 02: Property Graph Schema & Multi-Source Separation

Query 03 — Query [I08]: Deterministic Bridge TMDb / MovieLens

- **Appendix Reference:** `backend/scripts/import_movielenlens.cypher:L44`
- **Slide:** 02 · Property Graph Schema & Multi-Source Separation
- **Execution Frequency:** Ingestion pipeline, dataset refresh
- **Key Goal:** Ingest `links.csv` to map MovieLens integer IDs to external TMDb IDs, establishing the explicit bridge edge `[ :MATCHES_TMDB ]`.

```
LOAD CSV WITH HEADERS FROM 'file:///links.csv' AS row
CALL {
  WITH row
  MATCH (ml:MovieLensMovie {movieLensId: toInteger(row.movieId)})
  SET ml.imdbId = row.imdbId,
      ml.imdbFullId = CASE
        WHEN row.imdbId IS NULL OR trim(row.imdbId) = '' THEN null
        ELSE 'tt' + row.imdbId
      END
  WITH row, ml
  WHERE row.tmbdId IS NOT NULL AND trim(row.tmbdId) > ''
  MERGE (m:Movie {tmbdId: toInteger(row.tmbdId)})
  ON CREATE SET m.title = ml.title,
               m.source = 'tmbd_movielenlens_link',
               m.createdFromMovieLens = true
  MERGE (ml)-[:MATCHES_TMDB]->(m)
} IN TRANSACTIONS OF 1000 ROWS;
```

Line-by-Line Breakdown:

1. `LOAD CSV WITH HEADERS FROM 'file:///links.csv' AS row` * Streams `links.csv` from the server import directory. Each row is parsed into a dictionary where keys match CSV column headers (`movieId` , `imdbId` , `tmbdId`).
2. `CALL {` * Opens an atomic subquery block to enable periodic commits.
3. `WITH row` * Explicitly imports the `row` variable into the subquery's local scope.
4. `MATCH (ml:MovieLensMovie {movieLensId: toInteger(row.movieId)})` * Performs an indexed point lookup (`NodeIndexSeek`) on `:MovieLensMovie.movieLensId` . * `toInteger()` is mandatory because CSV values enter Cypher as string types (`"1"` vs `1`).
5. `SET ml.imdbId = row.imdbId,` * Stores the raw string IMDb identifier on the raw MovieLens node.
6. `ml.imdbFullId = CASE`
7. `WHEN row.imdbId IS NULL OR trim(row.imdbId) = '' THEN null`
8. `ELSE 'tt' + row.imdbId`
9. `END` * Sanitization logic: IMDb standard IDs require the `'tt'` prefix followed by a 7-digit zero-padded number. If the column is null or whitespace, it writes `null` to avoid storing empty strings.
10. `WITH row, ml`

- Carries both `row` and the populated `ml` node forward.
11. `WHERE row.tmbdId IS NOT NULL AND trim(row.tmbdId) <> ''`
 - Guardrail: skips MovieLens rows that do not have a corresponding TMDb identifier.
 12. `MERGE (m:Movie {tmbdId: toInteger(row.tmbdId)})`
 - Idempotent match-or-create on the canonical `:Movie` label using `tmbdId`.
 - *Concurrency & Multi-Mapping*: In MovieLens, multiple `movieId` values can link to the same `tmbdId` (e.g., director's cut vs theatrical release). `MERGE` guarantees that both MovieLens nodes map to the **same** canonical movie rather than throwing a duplicate key exception.
 13. `ON CREATE SET m.title = ml.title,`
 14. `m.source = 'tmbd_movie_lens_link',`
 15. `m.createdFromMovieLens = true`
 - Executed **only** if the `:Movie` node was created in this transaction. Sets initial title and provenance flags. If the movie already exists, existing properties remain untouched.
 16. `MERGE (ml)-[:MATCHES_TMDB]-(m)`
 - Creates the directed bridge relationship from analytical space to canonical space.
 17. `} IN TRANSACTIONS OF 1000 ROWS;`
 - Flushes the transaction to the WAL (Write-Ahead Log) every 1,000 records, releasing heap memory and avoiding JVM garbage collection pauses or `OutOfMemoryError`.

Query 04 — Schema DDL: Unique Constraints & Index Assertions

- **Appendix Reference:** `backend/neo4jService.js:L147`
- **Slide:** 02 · Property Graph Schema & Multi-Source Separation
- **Execution Frequency:** Server startup, migration scripts
- **Key Goal:** Enforce physical schema constraints guaranteeing entity integrity and enabling $O(1)$ B-Tree index seeks.

```
CREATE CONSTRAINT app_user_uid IF NOT EXISTS FOR (u:AppUser) REQUIRE u.uid IS UNIQUE;
CREATE CONSTRAINT app_user_email IF NOT EXISTS FOR (u:AppUser) REQUIRE u.emailNormalized IS UNIQUE;
CREATE CONSTRAINT movie_tmbd_id IF NOT EXISTS FOR (m:Movie) REQUIRE m.tmbdId IS UNIQUE;
CREATE CONSTRAINT movielens_movie_id IF NOT EXISTS FOR (ml:MovieLensMovie) REQUIRE ml.movieLensId IS UNIQUE;
CREATE CONSTRAINT movielens_user_id IF NOT EXISTS FOR (u:MovieLensUser) REQUIRE u.movieLensUserId IS UNIQUE;
CREATE CONSTRAINT genre_name IF NOT EXISTS FOR (g:Genre) REQUIRE g.name IS UNIQUE;
```

Line-by-Line Breakdown:

1. `CREATE CONSTRAINT app_user_uid IF NOT EXISTS FOR (u:AppUser) REQUIRE u.uid IS UNIQUE; *`
Enforces that every `AppUser` has a unique UUID `uid`. Automatically creates an underlying unique index. Lookups via `MATCH (u:AppUser {uid: $uid})` execute as single-lookup operations.
2. `CREATE CONSTRAINT app_user_email IF NOT EXISTS FOR (u:AppUser) REQUIRE u.emailNormalized IS UNIQUE; *`
Enforces unique normalized email addresses (lowercase, trimmed). Prevents duplicate registrations and race conditions during simultaneous user sign-ups.
3. `CREATE CONSTRAINT movie_tmbd_id IF NOT EXISTS FOR (m:Movie) REQUIRE m.tmbdId IS UNIQUE; *`
Guarantees canonical singularity for movies. Every movie in Agreeo is indexed by its TMDb integer ID.

4. `CREATE CONSTRAINT movielens_movie_id IF NOT EXISTS FOR (ml:MovieLensMovie) REQUIRE ml.movieLensId IS UNIQUE;` * Indexes historical MovieLens raw movies by their original integer ID.
5. `CREATE CONSTRAINT movielens_user_id IF NOT EXISTS FOR (u:MovieLensUser) REQUIRE u.movieLensUserId IS UNIQUE;` * Indexes historical MovieLens rating users.
6. `CREATE CONSTRAINT genre_name IF NOT EXISTS FOR (g:Genre) REQUIRE g.name IS UNIQUE;` * Guarantees that genres (e.g. "Sci-Fi", "Drama") are unique nodes. When movies connect via `[ :IN_GENRE ]`, they all point to the exact same shared vertex.

Architectural Significance:

- In Neo4j, `CREATE CONSTRAINT ... REQUIRE ... IS UNIQUE` is superior to a simple index because it acts as both a **database-level validation rule** (rejecting duplicate writes) and a **high-performance index** for query execution plans.

Slide 03: Set-Based Ingestion & Idempotent Mutation

Query 05 — Query [I09]: Micro-Batched Rating Ingestion

- **Appendix Reference:** `backend/scripts/import_movielens.cypher:L62`
- **Slide:** 03 · Set-Based Ingestion & Idempotent Mutation
- **Execution Frequency:** Initial data load, bulk analytical seeding
- **Key Goal:** Ingest 100k+ rating events connecting MovieLens users to movies in 10,000-row ACID micro-transactions.

```
LOAD CSV WITH HEADERS FROM 'file:///ratings.csv' AS row
CALL {
  WITH row
  MATCH (u:MovieLensUser {movieLensUserId: toInteger(row.userId)})
  MATCH (m:MovieLensMovie {movieLensId: toInteger(row.movieId)})
  MERGE (u)-[r:RATED]->(m)
  SET r.rating = toFloat(row.rating),
      r.timestamp = toInteger(row.timestamp),
      r.ratedAt = datetime({epochSeconds: toInteger(row.timestamp)})
} IN TRANSACTIONS OF 10000 ROWS;
```

Line-by-Line Breakdown:

1. `LOAD CSV WITH HEADERS FROM 'file:///ratings.csv' AS row` * Streams the large ratings dataset from disk. Columns: `userId`, `movieId`, `rating`, `timestamp`.
2. `CALL { WITH row` * Isolates each chunk into an independent transactional scope.
3. `MATCH (u:MovieLensUser {movieLensUserId: toInteger(row.userId)})` * Seeks the user node via `movielens_user_id` constraint ($O(1)$).
4. `MATCH (m:MovieLensMovie {movieLensId: toInteger(row.movieId)})` * Seeks the movie node via `movielens_movie_id` constraint ($O(1)$).
5. `MERGE (u)-[r:RATED]->(m)` * Ensures that re-running the script does not create duplicate rating edges between the same user and movie pair.
6. `SET r.rating = toFloat(row.rating),` * Casts the string rating (e.g. `"4.5"`) to an IEEE 754 float (`4.5`). Essential for downstream arithmetic calculations.
7. `r.timestamp = toInteger(row.timestamp),` * Preserves raw POSIX epoch seconds.

8. `r.ratedAt = datetime({epochSeconds: toInteger(row.timestamp)})` * Converts the epoch timestamp into a native Neo4j `ZonedDateTime` object. Enables temporal filtering (e.g., calculating age-based exponential decay).
9. `} IN TRANSACTIONS OF 10000 ROWS;` * Commits in batches of 10,000 edges. Balances throughput with transaction log overhead.

Query 06 — Query [M01]: Canonical Movie Upsert & Stale Edge Pruning

- **Appendix Reference:** `backend/movieRepository.js:L102`
- **Slide:** 03 · Set-Based Ingestion & Idempotent Mutation
- **Execution Frequency:** Daily background worker, TMDb webhook, dynamic movie hydration
- **Key Goal:** Perform non-destructive entity update on canonical movies and synchronize `:IN_GENRE` relationships without deleting shared genre vertices.

```
MERGE (m:Movie {tmdbId: $tmdbId})
SET m.title = coalesce($title, m.title),
    m.originalTitle = coalesce($originalTitle, m.originalTitle),
    m.overview = CASE WHEN $overview <> '' THEN $overview ELSE coalesce(m.overview, '') END,
    m.runtime = coalesce($runtime, m.runtime),
    m.tmdbHydrated = coalesce(m.tmdbHydrated, false) OR $tmdbHydrated
FOREACH (genreName IN $genres |
    MERGE (g:Genre {name: genreName})
    MERGE (m)-[:IN_GENRE]-(g)
)
WITH m
OPTIONAL MATCH (m)-[:stale:IN_GENRE]-(staleGenre:Genre)
FOREACH (relationship IN CASE
    WHEN size($genres) > 0 AND NOT staleGenre.name IN $genres THEN [stale]
    ELSE []
END | DELETE relationship);
```

Line-by-Line Breakdown:

1. `MERGE (m:Movie {tmdbId: $tmdbId})` * Locates or creates the target movie node using its primary key.
2. `SET m.title = coalesce($title, m.title),` * **Defensive coalesce pattern:** If the incoming `$title` parameter is `null`, it retains the current database value `m.title`, preventing accidental property erasure.
3. `m.originalTitle = coalesce($originalTitle, m.originalTitle),` * Preserves original untranslated title if update payload is missing it.
4. `m.overview = CASE WHEN $overview <> '' THEN $overview ELSE coalesce(m.overview, '') END,` * Guards against empty strings overwriting meaningful movie synopsis text.
5. `m.runtime = coalesce($runtime, m.runtime),` * Updates duration in minutes while preserving existing runtime.
6. `m.tmdbHydrated = coalesce(m.tmdbHydrated, false) OR $tmdbHydrated` * Boolean latch: once a movie is marked as fully hydrated from TMDb (`true`), it stays `true`.
7. `FOREACH (genreName IN $genres |` * Iterates through the list of genre names provided in the payload.
8. `MERGE (g:Genre {name: genreName})` * Idempotently asserts the existence of the shared `:Genre` node.
9. `MERGE (m)-[:IN_GENRE]-(g)`
10. `)`
 - Links the movie to the genre node.
11. `WITH m`
 - Passes the movie instance to the edge pruning pipeline.

12. `OPTIONAL MATCH (m)-[stale:IN_GENRE]→(staleGenre:Genre)`
 - Inspects all currently connected genre edges.
 13. `FOREACH (relationship IN CASE`
 14. `WHEN size($genres) > 0 AND NOT staleGenre.name IN $genres THEN [stale]`
 15. `ELSE []`
 16. `END | DELETE relationship);`
 - **Pruning Stale Edges Without Dropping Nodes:** If the incoming `$genres` list is non-empty, and an existing relationship points to a genre *not* in the updated list, that specific relationship `stale` is deleted. Notice that `DELETE relationship` deletes **only the edge**, keeping `staleGenre` intact for all other movies in the database!
-

Slide 04: Graph-Based Collaborative Filtering [M23]

Query 07 — Query [M23]: Master Multi-Hop Collaborative Filtering

- **Appendix Reference:** `backend/movieRepository.js:L897`
- **Slide:** 04 · Graph-Based Collaborative Filtering [M23]
- **Execution Frequency:** Home screen recommendation rail, user swipe deck generation
- **Key Goal:** Execute the 4-hop collaborative traversal with mathematical preference weighting, popularity damping, support shrinkage, and anti-join pattern negation.

```

MATCH (me:AppUser {uid: $uid})
// 1. Subquery: Extract Disliked Genres for Penalty
CALL {
  WITH me
  OPTIONAL MATCH (me)-[r:DISLIKED]→(m:Movie)
  OPTIONAL MATCH (m)←[:MATCHES_TMDB]-(ml:MovieLensMovie)-[:IN_GENRE]→(g1:Genre)
  OPTIONAL MATCH (m)-[:IN_GENRE]→(g2:Genre)
  WITH collect(DISTINCT g1.name) + collect(DISTINCT g2.name) AS rawGenres
  UNWIND rawGenres AS g
  RETURN collect(DISTINCT g) AS dislikedGenres
}
// 2. 4-Hop Traversal: Seed → MovieLens Similar Peers → Candidates
MATCH (me)-[signal:LIKED|SELECTED_FAVORITE|WATCHLISTED]→(seed:Movie)
MATCH (seed)←[:MATCHES_TMDB]-(seedML:MovieLensMovie)←[r1:RATED]-(similar:MovieLensUser)
WHERE r1.rating ≥ 4.0
WITH me, dislikedGenres, similar,
count(DISTINCT seed) AS overlapCount,
sum(
  CASE type(signal)
    WHEN 'SELECTED_FAVORITE' THEN coalesce(signal.weight, 4.0)
    WHEN 'LIKED' THEN 3.0
    WHEN 'WATCHLISTED' THEN 1.25
    ELSE 1.0
  END
  * exp(-0.005 * duration.inDays(coalesce(signal.createdAt, datetime()), datetime()).days)
  * (toFloat(r1.rating) - 3.0)
  * (1.0 / sqrt(log(toFloat(coalesce(seed.movieLensRatingCount, 0)) + 10.0)))
) AS similarityScore
WHERE similarityScore > 0
ORDER BY similarityScore DESC
LIMIT toInteger($neighborLimit)
// 3. Candidate Expansion & Anti-Join
MATCH (similar)-[r2:RATED]→(recML:MovieLensMovie)-[:MATCHES_TMDB]→(rec:Movie)
WHERE rec.tmbdId IS NOT NULL
AND NOT (me)-[:LIKED|DISLIKED|WATCHLISTED|ALREADY_SEEN|SELECTED_FAVORITE]→(rec)
WITH rec,
count(DISTINCT similar) AS similarUsers,
avg(toFloat(r2.rating)) AS avgSimilarRating,
sum(similarityScore * (toFloat(r2.rating) - 3.0)) / coalesce(sum(abs(similarityScore)), 1.0) AS w
WITH rec, similarUsers, avgSimilarRating,
(toFloat(similarUsers) / (toFloat(similarUsers) + 5.0)) AS supportWeight,
weightedPref
WITH rec, similarUsers, avgSimilarRating,
(weightedPref * supportWeight + (toFloat(coalesce(rec.movieLensAvgRating, 3.2)) - 3.0) * (1.0 - s
* log(similarUsers + 1.0) * 10.0 AS collaborativeScore
WHERE collaborativeScore > 0
RETURN rec.tmbdId AS tmbdId, rec.title AS title, collaborativeScore, similarUsers, avgSimilarRating
ORDER BY collaborativeScore DESC LIMIT 80;

```

Line-by-Line Breakdown:

Section 1: Subquery for Disliked Genres

1. `MATCH (me:AppUser {uid: $uid})` * Roots the traversal at the current authenticated user node.
2. `CALL { WITH me` * Opens an isolated subquery to aggregate the user's negative preferences.
3. `OPTIONAL MATCH (me)-[r:DISLIKED]→(m:Movie)` * Finds all movies the user explicitly swiped left on.
4. `OPTIONAL MATCH (m)←[:MATCHES_TMDB]-(ml:MovieLensMovie)-[:IN_GENRE]→(g1:Genre)`
5. `OPTIONAL MATCH (m)-[:IN_GENRE]→(g2:Genre)` * Traverses to genre vertices via both MovieLens mapping and direct TMDB edges.
6. `WITH collect(DISTINCT g1.name) + collect(DISTINCT g2.name) AS rawGenres` * Merges and deduplicates genre names across both catalogs.
7. `UNWIND rawGenres AS g`

8. `RETURN collect(DISTINCT g) AS dislikedGenres`
9. `}` * Returns a single array `dislikedGenres` containing names of genres with recorded dislikes.

Section 2: 4-Hop Peer Discovery & Mathematical Scoring

10. `MATCH (me)-[signal:LIKED|SELECTED_FAVORITE|WATCHLISTED]→(seed:Movie)`
 - **Hop 1:** Expands from the user to their positive interaction seeds.
11. `MATCH (seed)←[:MATCHES_TMDB]-(seedMl:MovieLensMovie)←[r1:RATED]-(similar:MovieLensUser)`
 - **Hops 2 & 3:** Traverses through the `MATCHES_TMDB` bridge to the MovieLens analytical node, then walks backward along `RATED` edges to discover peer users.
12. `WHERE r1.rating ≥ 4.0`
 - Pruning filter: only peers who gave a positive rating ($\geq 4.0/5.0$) to the shared movie qualify as taste neighbors.
13. `WITH me, dislikedGenres, similar, count(DISTINCT seed) AS overlapCount, sum(...) AS similarityScore`
 - Aggregates similarity metrics between the current user and each candidate peer.
14. `CASE type(signal) WHEN 'SELECTED_FAVORITE' THEN 4.0 WHEN 'LIKED' THEN 3.0 WHEN 'WATCHLISTED' THEN 1.25 ELSE 1.0 END`
 - **Action Weighting:** Explicit favorites carry the highest anchor weight (4.0), followed by likes (3.0) and watchlist saves (1.25).
15. `* exp(-0.005 * duration.inDays(coalesce(signal.createdAt, datetime()), datetime().days))`
 - **Exponential Temporal Decay:** Multiplies by $\exp(-\lambda \cdot t)$ where $\lambda = 0.005$. A swipe from yesterday has weight ≈ 1.0 ; a swipe from 180 days ago decays to ≈ 0.40 .
16. `* (toFloat(r1.rating) - 3.0)`
 - **Rating Centering:** Subtracts 3.0 from the peer's 5-star rating. A 5.0 rating yields +2.0, while a 4.0 yields +1.0.
17. `* (1.0 / sqrt(log(toFloat(coalesce(seed.movieLensRatingCount, 0)) + 10.0)))`
 - **Popularity Damping:** Penalizes ubiquitous blockbuster movies. Sharing a niche film with 20 ratings indicates vastly higher mutual taste alignment than sharing *The Avengers* with 50,000 ratings.
18. `WHERE similarityScore > 0 ORDER BY similarityScore DESC LIMIT toInteger($neighborLimit)`
 - Restricts downstream computation to the top K most correlated peers (e.g. 50 neighbors).

Section 3: Candidate Recommendation & Pattern Negation

19. `MATCH (similar)-[r2:RATED]→(recMl:MovieLensMovie)-[:MATCHES_TMDB]→(rec:Movie)`
 - **Hop 4:** Traverses from the selected similar peers to other movies they rated highly, and bridges back to canonical `Movie` vertices.
20. `WHERE rec.tmbdId IS NOT NULL`
 - Ensures the recommended movie has a valid playable record.
21. `AND NOT (me)-[:LIKED|DISLIKED|WATCHLISTED|ALREADY_SEEN|SELECTED_FAVORITE]→(rec)`
 - **Pattern Negation Anti-Join:** Graph engine filters out any movie the active user has already interacted with. Executed via fast edge-existence check.

22. `WITH rec, count(DISTINCT similar) AS similarUsers, avg(toFloat(r2.rating)) AS avgSimilarRating, sum( ... ) AS weightedPref`
 - Computes weighted average peer preference.
23. `WITH rec, similarUsers, avgSimilarRating, (toFloat(similarUsers) / (toFloat(similarUsers) + 5.0)) AS supportWeight, weightedPref`
 - **Bayesian Support Shrinkage:** Applies shrinkage factor $\frac{N}{N+5}$. If only 1 peer recommends the movie, $supportWeight = \frac{1}{6} \approx 0.16$. If 15 peers recommend it, $supportWeight = \frac{15}{20} = 0.75$.
24. `WITH rec, similarUsers, avgSimilarRating, (weightedPref * supportWeight + (rec.movieLensAvgRating - 3.0) * (1.0 - supportWeight)) * log(similarUsers + 1.0) * 10.0 AS collaborativeScore`
 - Blends peer preference with global Bayesian prior, scaled by $\log(N + 1)$.
25. `WHERE collaborativeScore > 0`
26. `RETURN rec.tmbdId AS tmbdId, rec.title AS title, collaborativeScore, similarUsers, avgSimilarRating`
27. `ORDER BY collaborativeScore DESC LIMIT 80;`
 - Emits top 80 ranked candidate movies.

Slide 05: Hybrid Ranking: Collaborative + Vector Search

Query 08 — Vector Index DDL Definition

- **Appendix Reference:** `backend/neo4jService.js:L194`
- **Slide:** 05 · Hybrid Ranking: Collaborative + Vector Search
- **Execution Frequency:** Initialization, schema migration
- **Key Goal:** Provision an HNSW (Hierarchical Navigable Small World) index for 384-dimensional dense embeddings on `:Tag` nodes.

```
CREATE VECTOR INDEX tag_embeddings IF NOT EXISTS
FOR (t:Tag) ON (t.embedding)
OPTIONS { indexConfig: {
  'vector.dimensions': 384,
  'vector.similarity_function': 'cosine'
}};
```

Line-by-Line Breakdown:

1. `CREATE VECTOR INDEX tag_embeddings IF NOT EXISTS` * Creates the named vector index `tag_embeddings`. `IF NOT EXISTS` ensures idempotent startup execution.
2. `FOR (t:Tag) ON (t.embedding)` * Targets the `embedding` property of nodes bearing the `:Tag` label.
3. `OPTIONS { indexConfig: {` * Configuration dictionary for Lucene/HNSW index driver.
4. `'vector.dimensions': 384,` * Sets embedding vector dimensionality to 384, matching the output format of the `multilingual-e5-small` embedding model.
5. `'vector.similarity_function': 'cosine'`
6. `}};` * Uses Cosine Similarity: $\cos(\theta) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}$. Ideal for normalized text embeddings where directional angle captures semantic meaning regardless of magnitude.

Query 09 — Query [M24 / E02]: Semantic Mood Search Traversal

- **Appendix Reference:** `backend/movieRepository.js:L975`
- **Slide:** 05 · Hybrid Ranking: Collaborative + Vector Search
- **Execution Frequency:** Mood search rail, vector candidate generation
- **Key Goal:** Query the vector index for nearest semantic tags, then walk the graph to surface unviewed movies weighted by tag frequency and Inverse Document Frequency (IDF).

```
CALL db.index.vector.queryNodes('tag_embeddings', 15, $targetEmbedding)
YIELD node AS tag, score
MATCH (tag)←[ht:HAS_TAG]-(mlm:MovieLensMovie)-[:MATCHES_TMDB]→(m:Movie)
WHERE NOT (u:AppUser {uid: $uid})→(m)
RETURN m.title AS title, m.tmdbId AS tmdbId,
       sum(score * ht.frequency * tag.idf) AS semanticScore
ORDER BY semanticScore DESC LIMIT 20;
```

Line-by-Line Breakdown:

1. `CALL db.index.vector.queryNodes('tag_embeddings', 15, $targetEmbedding)` * Searches the HNSW graph index for the 15 nearest tag nodes relative to `$targetEmbedding`.
2. `YIELD node AS tag, score` * Yields each matched `:Tag` node and its cosine similarity `score` ($\in [0, 1]$).
3. `MATCH (tag)←[ht:HAS_TAG]-(mlm:MovieLensMovie)-[:MATCHES_TMDB]→(m:Movie)` * Navigates from the vector tag through MovieLens tagged movies to canonical application movies.
4. `WHERE NOT (u:AppUser {uid: $uid})→(m)` * Filters out movies already connected by any outbound edge from the current user (seen, liked, disliked).
5. `RETURN m.title AS title, m.tmdbId AS tmdbId, sum(score * ht.frequency * tag.idf) AS semanticScore` * Computes the semantic relevance score:

$$\text{Score} = \sum_{\text{tag}} \text{Similarity} \times \text{Frequency} \times \text{IDF}$$

* Highly specific tags (high IDF) contribute far more than ubiquitous generic descriptors.

6. `ORDER BY semanticScore DESC LIMIT 20;` * Returns the top 20 movies matching the semantic mood vector.

Query 10 — Reciprocal Rank Fusion (RRF) Merge Logic

- **Appendix Reference:** `backend/movieRepository.js:L1040`
- **Slide:** 05 · Hybrid Ranking: Collaborative + Vector Search
- **Execution Frequency:** Hybrid pipeline rank arbitration
- **Key Goal:** Merge rankings from heterogeneous sources (collaborative filtering vs vector similarity) without numerical scale distortion.

```
// Reciprocal Rank Fusion between Graph and Vector ranks:
// RRF(m) = (w_graph / (60 + rank_graph)) + (w_vector / (60 + rank_vector))
WITH candidate,
  coalesce(0.65 / (60.0 + candidate.rankCollab), 0.0) +
  coalesce(0.35 / (60.0 + candidate.rankVector), 0.0) AS rrfScore
RETURN candidate.tmdbId AS id, candidate.title AS title, rrfScore
ORDER BY rrfScore DESC LIMIT 20;
```

Line-by-Line Breakdown:

1. `// Reciprocal Rank Fusion between Graph and Vector ranks:`
2. `// RRF(m) = (w_graph / (60 + rank_graph)) + (w_vector / (60 + rank_vector))` * Documents the classical RRF formula parameterized with constant $k = 60$.
3. `WITH candidate,` * Receives unified candidate objects containing `rankCollab` and `rankVector`.
4. `coalesce(0.65 / (60.0 + candidate.rankCollab), 0.0) +` * Assigns 65% weight to collaborative filtering rank. If the movie was not present in the collaborative pool, `coalesce` yields `0.0`.
5. `coalesce(0.35 / (60.0 + candidate.rankVector), 0.0) AS rrfScore` * Assigns 35% weight to semantic vector rank.
6. `RETURN candidate.tmdbId AS id, candidate.title AS title, rrfScore`
7. `ORDER BY rrfScore DESC LIMIT 20;` * Projects final hybrid ranking sorted by unified RRF score.

Slide 06: Atomic Daily Interaction Transaction [M06]

Query 11 — Query [M06]: Atomic Daily Quota Increment

- **Appendix Reference:** `backend/movieRepository.js:L376`
- **Slide:** 06 · Atomic Daily Interaction Transaction [M06]
- **Execution Frequency:** On every card swipe (Like, Dislike, Watchlist, Skip)
- **Key Goal:** Atomically manage and increment daily user swipe rate limits inside database transactions, preventing race conditions across concurrent client calls.

```
MATCH (u:AppUser {uid: $uid})
MERGE (quota:DailySwipeQuota {key: $quotaKey})
ON CREATE SET quota.uid = $uid,
               quota.day = date($day),
               quota.used = 0,
               quota.createdAt = datetime()
MERGE (u)-[:HAS_DAILY_QUOTA]->(quota)
SET quota.used = coalesce(quota.used, 0) + $increment
RETURN quota.used AS usedToday;
```

Line-by-Line Breakdown:

1. `MATCH (u:AppUser {uid: $uid})` * Matches target authenticated user.
2. `MERGE (quota:DailySwipeQuota {key: $quotaKey})` * `$quotaKey` is formed as `"<uid>_<YYYY-MM-DD>"`. Idempotently creates or matches the quota entity for the active calendar day.
3. `ON CREATE SET quota.uid = $uid, quota.day = date($day), quota.used = 0, quota.createdAt = datetime()` * Initializes quota counters upon first swipe of the day.
4. `MERGE (u)-[:HAS_DAILY_QUOTA]->(quota)` * Connects user to their daily quota tracking node.
5. `SET quota.used = coalesce(quota.used, 0) + $increment` * **Atomic In-Place Increment:** The write lock acquired on `quota` guarantees that concurrent requests cannot cause lost updates.
6. `RETURN quota.used AS usedToday;` * Returns current count to enforce client rate-limiting.

Query 12 — Query [M08]: Invariant State Cleanup (Mutual Exclusion)

- **Appendix Reference:** `backend/movieRepository.js:L415`

- **Slide:** 06 · Atomic Daily Interaction Transaction [M06]
- **Execution Frequency:** Inside swipe transaction before writing new action edge
- **Key Goal:** Enforce mutual exclusion. A user cannot simultaneously `LIKE` and `DISLIKE` the same movie.

```
MATCH (u:AppUser {uid: $uid})-[r:LIKED|DISLIKED|WATCHLISTED|ALREADY_SEEN]→(m:Movie {tmdbId: $tmdbId})
WHERE type(r) <> $newAction
DELETE r;
```

Line-by-Line Breakdown:

1. `MATCH (u:AppUser {uid: $uid})-[r:LIKED|DISLIKED|WATCHLISTED|ALREADY_SEEN]→(m:Movie {tmdbId: $tmdbId})` * Inspects existing interaction edges between the user and the movie.
2. `WHERE type(r) <> $newAction` * Filters for edges whose relationship type differs from the incoming action (e.g. existing `DISLIKED` when incoming is `LIKED`).
3. `DELETE r;` * Atomically drops stale contradictory relationships.

Query 13 — Query [M09]: Materialize Action Edge

- **Appendix Reference:** `backend/movieRepository.js:L442`
- **Slide:** 06 · Atomic Daily Interaction Transaction [M06]
- **Execution Frequency:** Inside swipe transaction immediately following cleanup
- **Key Goal:** Create the new interaction edge with audit timestamp.

```
MATCH (u:AppUser {uid: $uid}), (m:Movie {tmdbId: $tmdbId})
MERGE (u)-[r:LIKED]→(m)
SET r.createdAt = datetime();
```

Line-by-Line Breakdown:

1. `MATCH (u:AppUser {uid: $uid}), (m:Movie {tmdbId: $tmdbId})` * Seeks both user and movie vertices via unique index constraints.
2. `MERGE (u)-[r:LIKED]→(m)` * Idempotently materializes the edge.
3. `SET r.createdAt = datetime();` * Records exact interaction timestamp, driving temporal decay in Query M23.

Slide 07: Social Graph: Symmetric Friendships & Invariants [S06]

Query 14 — Query [S06]: Atomic Friendship Acceptance & Block Invariant

- **Appendix Reference:** `backend/socialRepository.js:L197`
- **Slide:** 07 · Social Graph: Symmetric Friendships & Invariants [S06]
- **Execution Frequency:** User accepts friend request
- **Key Goal:** Atomically accept friendship, enforce bidirectional blocking invariants, and create an undirected relationship stored once in memory.

```

MATCH (from:AppUser {uid: $uid})
MATCH (to:AppUser {uid: $targetUserId})
WHERE from.uid <> to.uid
  AND NOT (from)-[:FRIEND]-(to)
  AND NOT (from)-[:BLOCKED]->(to)
  AND NOT (to)-[:BLOCKED]->(from)
MATCH (to)-[r:SENT_FRIEND_REQUEST {status: 'pending'}]->(from)
SET r.status = 'accepted', r.updatedAt = datetime()
MERGE (from)-[a:FRIEND]-(to) // Undirected relationship: stored ONCE in memory
ON CREATE SET a.createdAt = datetime()
RETURN r.requestId;

```

Line-by-Line Breakdown:

1. `MATCH (from:AppUser {uid: $uid})`
2. `MATCH (to:AppUser {uid: $targetUserId})` * Matches both user entities.
3. `WHERE from.uid <> to.uid` * Self-friendship invariant: users cannot friend themselves.
4. `AND NOT (from)-[:FRIEND]-(to)` * Prevents redundant duplicate processing.
5. `AND NOT (from)-[:BLOCKED]->(to)`
6. `AND NOT (to)-[:BLOCKED]->(from)` * **Bidirectional Block Invariant:** Guarantees that friendships cannot be formed if either user has blocked the other.
7. `MATCH (to)-[r:SENT_FRIEND_REQUEST {status: 'pending'}]->(from)` * Validates that an authentic, pending request exists from `to` to `from`.
8. `SET r.status = 'accepted', r.updatedAt = datetime()` * Updates request status to accepted.
9. `MERGE (from)-[a:FRIEND]-(to)` * **Undirected Edge Storage Optimization:** Notice the omission of direction arrow `>` or `<`. In Neo4j, querying `(a)-[:FRIEND]-(b)` treats the relationship as bidirectional while storing only **one physical relationship record** in the pointer list, cutting memory overhead in half.
10. `ON CREATE SET a.createdAt = datetime()`
 - Sets friendship inception timestamp.
11. `RETURN r.requestId;`
 - Emits confirmed request ID.

Query 15 — Query [S01]: Send Friend Request

- **Appendix Reference:** `backend/socialRepository.js:L64`
- **Slide:** 07 · Social Graph: Symmetric Friendships & Invariants [S06]
- **Execution Frequency:** User sends friend request
- **Key Goal:** Dispatch a directed pending friendship invitation adhering to blocking rules.

```

MATCH (from:AppUser {uid: $fromUid}), (to:AppUser {uid: $toUid})
WHERE from <> to
  AND NOT (from)-[:FRIEND]-(to)
  AND NOT (from)-[:BLOCKED]->(to)
  AND NOT (to)-[:BLOCKED]->(from)
MERGE (from)-[r:SENT_FRIEND_REQUEST {requestId: $reqId}]->(to)
ON CREATE SET r.status = 'pending', r.createdAt = datetime();

```

Line-by-Line Breakdown:

1. `MATCH (from:AppUser {uid: $fromUid}), (to:AppUser {uid: $toUid})` * Seeks sender and recipient users.

2. `WHERE from <> to AND NOT (from)-[:FRIEND]-(to) AND NOT (from)-[:BLOCKED]->(to) AND NOT (to)-[:BLOCKED]->(from)` * Complete invariant pre-flight check.
3. `MERGE (from)-[r:SENT_FRIEND_REQUEST {requestId: $reqId}]->(to)` * Creates the directed relationship pointing from requester to target.
4. `ON CREATE SET r.status = 'pending', r.createdAt = datetime();` * Sets initial pending status.

Slide 08: Movie Night: Group Compatibility & Concurrent Voting [S40 / S44]

Query 16 — Query [S40]: Group Social Compatibility Matrix

- **Appendix Reference:** `backend/socialRepository.js:L1443`
- **Slide:** 08 · Movie Night: Group Compatibility & Concurrent Voting
- **Execution Frequency:** Movie Night session setup, shortlist candidate evaluation
- **Key Goal:** Evaluate preference states for N users across K candidate movies in a single query to construct the group decision matrix.

```
MATCH (u:AppUser)
WHERE u.uid IN $userIds
MATCH (m:Movie)
WHERE m.tmdbId IN $candidateIds
OPTIONAL MATCH (u)-[liked:LIKED]->(m)
OPTIONAL MATCH (u)-[disliked:DISLIKED]->(m)
OPTIONAL MATCH (u)-[watchlisted:WATCHLISTED]->(m)
OPTIONAL MATCH (u)-[seen:ALREADY_SEEN]->(m)
OPTIONAL MATCH (u)-[rated:RATED_APP]->(m)
RETURN u.uid AS userId,
       m.tmdbId AS tmdbId,
       count(liked) > 0 AS liked,
       count(disliked) > 0 AS disliked,
       count(watchlisted) > 0 AS inWatchlist,
       count(seen) > 0 AS watched,
       max(rated.rating) AS rating;
```

Line-by-Line Breakdown:

1. `MATCH (u:AppUser) WHERE u.uid IN $userIds` * Matches all participating users in the group session via index in-list operator.
2. `MATCH (m:Movie) WHERE m.tmdbId IN $candidateIds` * Matches candidate movies in shortlist.
3. `OPTIONAL MATCH (u)-[liked:LIKED]->(m)`
4. `OPTIONAL MATCH (u)-[disliked:DISLIKED]->(m)`
5. `OPTIONAL MATCH (u)-[watchlisted:WATCHLISTED]->(m)`
6. `OPTIONAL MATCH (u)-[seen:ALREADY_SEEN]->(m)`
7. `OPTIONAL MATCH (u)-[rated:RATED_APP]->(m)` * Evaluates individual interaction edges for every user-movie pair.
8. `RETURN u.uid AS userId, m.tmdbId AS tmdbId, count(liked) > 0 AS liked, count(disliked) > 0 AS disliked, count(watchlisted) > 0 AS inWatchlist, count(seen) > 0 AS watched, max(rated.rating) AS rating;` * Emits boolean flags and ratings. The backend matrix engine uses these values to calculate group compatibility (e.g. penalizing movies disliked by any member, rewarding watchlist overlaps).

Query 17 — Query [S44]: Exclusive Write Lock on Event Node for Voting

- **Appendix Reference:** `backend/socialRepository.js:L1645`
- **Slide:** 08 · Movie Night: Group Compatibility & Concurrent Voting
- **Execution Frequency:** User casts vote during live voting round
- **Key Goal:** Acquire an exclusive write lock on the shared `MovieNight` parent node, serializing simultaneous votes from participants and eliminating race conditions.

```
MATCH (user:AppUser {uid: $uid})-[part:PARTICIPATES_IN]→(event:MovieNight {id: $eventId})
MATCH (event)-[rel:HAS_CANDIDATE]→(movie:Movie {tmdbId: $tmdbId})
WHERE part.status = 'joined'
  AND (rel.eliminated IS NULL OR NOT rel.eliminated)
MERGE (user)-[v:VOTED_IN {eventId: $eventId}]→(movie)
ON CREATE SET v.createdAt = datetime()
SET v.vote = $vote,
    v.updatedAt = datetime(),
    event.status = 'voting',
    event.updatedAt = datetime() // ACQUIRES EXCLUSIVE WRITE LOCK ON MOVIE NIGHT NODE!
RETURN movie.tmdbId AS tmdbId
LIMIT 1;
```

Line-by-Line Breakdown:

1. `MATCH (user:AppUser {uid: $uid})-[part:PARTICIPATES_IN]→(event:MovieNight {id: $eventId})` *
Verifies user participation in target event.
2. `MATCH (event)-[rel:HAS_CANDIDATE]→(movie:Movie {tmdbId: $tmdbId})` * Matches movie within the event's candidate pool.
3. `WHERE part.status = 'joined' AND (rel.eliminated IS NULL OR NOT rel.eliminated)` * Restricts voting to active members on non-eliminated candidate films.
4. `MERGE (user)-[v:VOTED_IN {eventId: $eventId}]→(movie)` * Scopes the vote relationship to this specific event ID.
5. `ON CREATE SET v.createdAt = datetime()`
6. `SET v.vote = $vote, v.updatedAt = datetime(),` * Records or updates vote value.
7. `event.status = 'voting',`
8. `event.updatedAt = datetime() // ACQUIRES EXCLUSIVE WRITE LOCK ON MOVIE NIGHT NODE!` *
Pessimistic Concurrency Serialization: Modifying property `event.updatedAt` forces Neo4j's transaction manager to acquire an **exclusive write lock** on the `event` node. Any other simultaneous voting queries for the same event must wait in a FIFO queue until this transaction commits. This prevents race conditions during vote tallying and round finalization.
9. `RETURN movie.tmdbId AS tmdbId LIMIT 1;` * Emits confirmation.

Slide 09: Query Engineering: Cardinality, Indexes & Execution Plans

Query 18 — Query [M22]: Subquery Isolation (Anti-Cartesian Library Query)

- **Appendix Reference:** `backend/movieRepository.js:L847`
- **Slide:** 09 · Query Engineering: Cardinality, Indexes & Execution Plans
- **Execution Frequency:** User library load, profile state hydration
- **Key Goal:** Fetch liked, disliked, watchlisted, and seen collections using isolated subqueries, keeping intermediate cardinality at 1 row and preventing a 1,000,000-row Cartesian explosion.

```

MATCH (u:AppUser {uid: $uid})
CALL {
  WITH u
  MATCH (u)-[:LIKED]→(m:Movie)
  RETURN collect(m.tmbdId) AS likedIds
}
CALL {
  WITH u
  MATCH (u)-[:DISLIKED]→(m:Movie)
  RETURN collect(m.tmbdId) AS dislikedIds
}
CALL {
  WITH u
  MATCH (u)-[:WATCHLISTED]→(m:Movie)
  RETURN collect(m.tmbdId) AS watchlistIds
}
CALL {
  WITH u
  MATCH (u)-[:ALREADY_SEEN]→(m:Movie)
  RETURN collect(m.tmbdId) AS seenIds
}
RETURN likedIds, dislikedIds, watchlistIds, seenIds;

```

Line-by-Line Breakdown:

1. `MATCH (u:AppUser {uid: $uid})` * Matches single target user (Cardinality = 1).
2. `CALL { WITH u MATCH (u)-[:LIKED]→(m:Movie) RETURN collect(m.tmbdId) AS likedIds }` *
Subquery 1: Expands liked movies, rolls them into an in-memory list `likedIds`, and returns **1 scalar row**.
3. `CALL { WITH u MATCH (u)-[:DISLIKED]→(m:Movie) RETURN collect(m.tmbdId) AS dislikedIds }` *
Subquery 2: Expands disliked movies, aggregates into `dislikedIds`, returns **1 scalar row**.
4. `CALL { WITH u MATCH (u)-[:WATCHLISTED]→(m:Movie) RETURN collect(m.tmbdId) AS watchlistIds }` *
Subquery 3: Expands watchlist movies, aggregates into `watchlistIds`, returns **1 scalar row**.
5. `CALL { WITH u MATCH (u)-[:ALREADY_SEEN]→(m:Movie) RETURN collect(m.tmbdId) AS seenIds }` *
Subquery 4: Expands seen movies, aggregates into `seenIds`, returns **1 scalar row**.
6. `RETURN likedIds, dislikedIds, watchlistIds, seenIds;` * Emits the 4 discrete lists in a single round-trip.

Query 19 — Cartesian Explosion Anti-Pattern (What NOT to do)

- **Appendix Reference:** Theoretical Contrast
- **Slide:** 09 · Query Engineering: Cardinality, Indexes & Execution Plans
- **Execution Frequency:** Educational anti-pattern for academic comparison
- **Key Goal:** Demonstrate why chaining naive `OPTIONAL MATCH` clauses causes exponential cardinality explosion.

```

// ANTI-PATTERN: Produces L * D * W * S intermediate rows!
MATCH (u:AppUser {uid: $uid})
OPTIONAL MATCH (u)-[:LIKED]→(m1:Movie)
OPTIONAL MATCH (u)-[:DISLIKED]→(m2:Movie)
OPTIONAL MATCH (u)-[:WATCHLISTED]→(m3:Movie)
OPTIONAL MATCH (u)-[:ALREADY_SEEN]→(m4:Movie)
RETURN collect(m1), collect(m2), collect(m3), collect(m4);

```

Line-by-Line Breakdown:

1. `// ANTI-PATTERN: Produces L * D * W * S intermediate rows!`
2. `MATCH (u:AppUser {uid: $uid})` * User root: 1 row.

3. `OPTIONAL MATCH (u)-[:LIKED]→(m1:Movie)` * If user liked 100 movies, stream becomes **100 rows**.
4. `OPTIONAL MATCH (u)-[:DISLIKED]→(m2:Movie)` * If user disliked 20 movies, stream explodes to $100 \times 20 = 2,000$ rows.
5. `OPTIONAL MATCH (u)-[:WATCHLISTED]→(m3:Movie)` * If user saved 50 movies to watchlist, stream explodes to $2,000 \times 50 = 100,000$ rows.
6. `OPTIONAL MATCH (u)-[:ALREADY_SEEN]→(m4:Movie)` * If user marked 10 movies seen, stream explodes to $100,000 \times 10 = 1,000,000$ intermediate rows in JVM RAM!
7. `RETURN collect(m1), collect(m2), collect(m3), collect(m4);` * The database runs out of memory attempting to deduplicate 1 million rows that only contain 180 actual movies.

Slide 10: Integrity, Security & Honest Architectural Trade-offs

Query 20 — Security: Parameterized Execution vs Cypher Injection

- **Appendix Reference:** `backend/authController.js:L234`
- **Slide:** 10 · Integrity, Security & Honest Architectural Trade-offs
- **Execution Frequency:** User authentication, password verification
- **Key Goal:** Demonstrate parameterized query execution preventing Cypher injection.

```
// SECURE: Fully parameterized Cypher query
MATCH (u:AppUser {emailNormalized: $emailNormalized})
RETURN u.uid AS uid,
       u.email AS email,
       u.passwordHash AS passwordHash,
       u.onboardingCompleted AS onboardingCompleted;
```

Line-by-Line Breakdown:

1. `// SECURE: Fully parameterized Cypher query`
2. `MATCH (u:AppUser {emailNormalized: $emailNormalized})` * `$emailNormalized` is supplied through the driver's binary parameters map. Even if the user submits `' OR 1=1 OR ''=''`, the string is treated purely as literal data and cannot alter query AST structure.
3. `RETURN u.uid AS uid, u.email AS email, u.passwordHash AS passwordHash, u.onboardingCompleted AS onboardingCompleted;` * Emits user authentication payload.

Query 21 — Defensive UNWIND Pattern (Avoiding Pipeline Abort)

- **Appendix Reference:** `Cypher Best Practice`
- **Slide:** 10 · Integrity, Security & Honest Architectural Trade-offs
- **Execution Frequency:** Batch mutations with potentially empty input lists
- **Key Goal:** Prevent empty list parameters from silently terminating downstream Cypher pipelines.

```
// DEFENSIVE PATTERN: Protects pipeline when $incomingList is empty
WITH $incomingList AS list
UNWIND (CASE WHEN size(list) > 0 THEN list ELSE [null] END) AS item
WITH item WHERE item IS NOT NULL
MATCH (m:Movie {tmdbId: item.id})
// Subsequent mutations execute safely without being dropped!
MERGE (m)-[:TAGGED]→(...);
```

Line-by-Line Breakdown:

1. `// DEFENSIVE PATTERN: Protects pipeline when $incomingList is empty`
2. `WITH $incomingList AS list` * Receives incoming array.
3. `UNWIND (CASE WHEN size(list) > 0 THEN list ELSE [null] END) AS item` * **The UNWIND Pitfall:** In standard Cypher, `UNWIND [] AS item` produces **0 rows**, instantly aborting the entire remainder of the query! The `CASE` expression replaces an empty list with `[null]`, guaranteeing that at least 1 row continues downstream.
4. `WITH item WHERE item IS NOT NULL` * Filters out the dummy null record cleanly.
5. `MATCH (m:Movie {tmdbId: item.id})`
6. `MERGE (m)-[:TAGGED]->( ... );` * Executes without silent abort failures.

Slide 11: Takeaway & Live Demo

Query 22 — Preset [P05]: Bridge TMDb / MovieLens Verification

- **Appendix Reference:** `Live Demo Preset`
- **Slide:** 11 · Takeaway & Live Demo
- **Execution Frequency:** Live demo verification, sanity probe
- **Key Goal:** Visually inspect active bridge edges connecting MovieLens analytical nodes to canonical TMDb movie nodes.

```
MATCH (ml:MovieLensMovie)-[:MATCHES_TMDB]->(m:Movie)
RETURN m.title AS tmdbTitle, m.tmdbId AS tmdbId, ml.movieLensId AS mlId
LIMIT 10;
```

Line-by-Line Breakdown:

1. `MATCH (ml:MovieLensMovie)-[:MATCHES_TMDB]->(m:Movie)` * Traverses bridge edges across schema boundaries.
2. `RETURN m.title AS tmdbTitle, m.tmdbId AS tmdbId, ml.movieLensId AS mlId` * Projects both canonical and analytical IDs alongside movie title.
3. `LIMIT 10;` * Restricts output to 10 rows for instant live demonstration.

Query 23 — Preset [P03]: Didactic Collaborative Traversal with PROFILE

- **Appendix Reference:** `Live Demo Preset`
- **Slide:** 11 · Takeaway & Live Demo
- **Execution Frequency:** Live demo execution plan demonstration
- **Key Goal:** Execute live 4-hop collaborative traversal with execution profiler enabled to prove low DbHits and absence of Cartesian products.

```

PROFILE
MATCH (me:AppUser)-[:LIKED]→(:Movie)←[:MATCHES_TMDB]-(seed:MovieLensMovie)
MATCH (seed)←[r1:RATED]-(sim:MovieLensUser)-[r2:RATED]→(rec:MovieLensMovie)-[:MATCHES_TMDB]→(m:Movie)
WHERE r1.rating ≥ 4.0 AND r2.rating ≥ 4.0
AND NOT (me)-[:LIKED|ALREADY_SEEN]→(m)
RETURN m.title AS suggestion,
       count(DISTINCT sim) AS similarUsers,
       round(avg(r2.rating), 2) AS avgRating
ORDER BY similarUsers DESC, avgRating DESC
LIMIT 10;

```

Line-by-Line Breakdown:

1. `PROFILE` * Directs Neo4j to execute the query and generate an interactive execution plan displaying actual row counts, operator pipelines, and total `DbHits` (disk/memory page accesses).
2. `MATCH (me:AppUser)-[:LIKED]→(:Movie)←[:MATCHES_TMDB]-(seed:MovieLensMovie)` * Expands seed movies from user's likes across bridge.
3. `MATCH (seed)←[r1:RATED]-(sim:MovieLensUser)-[r2:RATED]→(rec:MovieLensMovie)-[:MATCHES_TMDB]→(m:Movie)` * Traverses to peers and candidate movies.
4. `WHERE r1.rating ≥ 4.0 AND r2.rating ≥ 4.0` * Restricts both rating hops to positive scores.
5. `AND NOT (me)-[:LIKED|ALREADY_SEEN]→(m)` * Anti-join filtering.
6. `RETURN m.title AS suggestion, count(DISTINCT sim) AS similarUsers, round(avg(r2.rating), 2) AS avgRating` * Projects aggregate peer support.
7. `ORDER BY similarUsers DESC, avgRating DESC LIMIT 10;` * Sorts by consensus.

Query 24 — Preset [P04]: Degree Centrality & Genre Aggregation

- **Appendix Reference:** [Live Demo Preset](#)
- **Slide:** 11 · Takeaway & Live Demo
- **Execution Frequency:** Live demo topology analytics
- **Key Goal:** Calculate in-degree centrality of genre hubs across the canonical movie catalog.

```

MATCH (g:Genre)←[:IN_GENRE]-(m:Movie)
RETURN g.name AS genre, count(m) AS movieCount
ORDER BY movieCount DESC LIMIT 10;

```

Line-by-Line Breakdown:

1. `MATCH (g:Genre)←[:IN_GENRE]-(m:Movie)` * Walks incoming `IN_GENRE` relationships from all canonical movies into genre hub nodes.
2. `RETURN g.name AS genre, count(m) AS movieCount` * Counts the in-degree (number of connected movies) per genre.
3. `ORDER BY movieCount DESC LIMIT 10;` * Ranks the top 10 most populated genres in the database.

Complete Reference Matrix (All 24 Queries)

#	Slide	Name / Operation	Key Pattern / Mechanism	Primary Neo4j Labels Involved
01	01	APOC Graph Sizing & Density Inspection	<code>apoc.meta.stats()</code> $O(1)$ store metadata read	Global graph store
02	01	Dual-Population Graph Census	In-memory count aggregation with <code>WITH</code> isolation	<code>:AppUser</code> , <code>:MovieLensUser</code> , <code>:Movie</code>
03	02	Deterministic Bridge TMDB / MovieLens	<code>LOAD CSV</code> + subquery batching + <code>MERGE</code>	<code>:MovieLensMovie</code> , <code>:Movie</code>
04	02	Schema Constraints & Indexes	DDL <code>CREATE CONSTRAINT ... REQUIRE ... IS UNIQUE</code>	<code>:AppUser</code> , <code>:Movie</code> , <code>:MovieLensMovie</code> , <code>:Genre</code>
05	03	Micro-Batched Rating Ingestion	<code>CALL { ... } IN TRANSACTIONS OF 10000 ROWS</code>	<code>:MovieLensUser</code> , <code>:MovieLensMovie</code>
06	03	Movie Upsert & Stale Edge Pruning	Coalesce defensive mutations + selective edge delete	<code>:Movie</code> , <code>:Genre</code>
07	04	Master Collaborative Filtering [M23]	4-hop traversal, temporal decay, shrinkage, anti-join	<code>:AppUser</code> , <code>:Movie</code> , <code>:MovieLensMovie</code> , <code>:MovieLensUser</code>
08	05	Vector Index DDL Definition	HNSW index on 384-dimensional cosine vector space	<code>:Tag {embedding}</code>
09	05	Semantic Mood Search Traversal [M24]	<code>CALL db.index.vector.queryNodes</code> + IDF graph walk	<code>:Tag</code> , <code>:MovieLensMovie</code> , <code>:Movie</code>
10	05	Reciprocal Rank Fusion (RRF) Merge	Rank-based fusion: $\sum \frac{w}{60 + \text{rank}}$	In-memory candidates
11	06	Atomic Daily Quota Increment [M06]	In-place atomic counter mutation on daily quota node	<code>:AppUser</code> , <code>:DailySwipeQuota</code>
12	06	Invariant State Cleanup (Mutual Exclusion)	Multi-rel match + type inequality deletion	<code>:AppUser</code> , <code>:Movie</code>
13	06	Materialize Action Edge [M09]	Idempotent <code>MERGE</code> with creation timestamp	<code>:AppUser</code> , <code>:Movie</code>
14	07	Atomic Friendship Acceptance [S06]	Undirected single-pointer <code>[:FRIEND]</code> + block invariants	<code>:AppUser</code>
15	07	Send Friend Request [S01]	Directed request creation with block pre-flight	<code>:AppUser</code>
16	08	Group Social Compatibility Matrix [S40]	Batch in-list match across users and movies	<code>:AppUser</code> , <code>:Movie</code>
17	08	Exclusive Write Lock on Event Node [S44]	Property mutation on parent node for FIFO serialization	<code>:AppUser</code> , <code>:MovieNight</code> , <code>:Movie</code>

#	Slide	Name / Operation	Key Pattern / Mechanism	Primary Neo4j Labels Involved
18	09	Subquery Isolation (Anti-Cartesian) [M22]	4 independent <code>CALL { WITH u ... }</code> subqueries	<code>:AppUser</code> , <code>:Movie</code>
19	09	Cartesian Explosion Anti-Pattern	Multiple chained <code>OPTIONAL MATCH</code> multiplying rows	<code>:AppUser</code> , <code>:Movie</code>
20	10	Parameterized Execution vs Injection	Binary parameter mapping preventing Cypher AST injection	<code>:AppUser</code>
21	10	Defensive UNWIND Pattern	<code>CASE WHEN size(list)>0 ... ELSE [null]</code> wrapper	<code>:Movie</code>
22	11	Live Bridge Verification [P05]	Live traversal across <code>[:MATCHES_TMDB]</code>	<code>:MovieLensMovie</code> , <code>:Movie</code>
23	11	Didactic Collaborative Traversal [P03]	<code>PROFILE</code> execution plan demonstrating Index-Free Adjacency	<code>:AppUser</code> , <code>:MovieLensUser</code> , <code>:Movie</code>
24	11	Degree Centrality & Genre Aggregation [P04]	In-degree aggregation on shared genre hubs	<code>:Genre</code> , <code>:Movie</code>

Compiled by Agreeo Architecture Team for Academic Examination & Systems Defense.